Avaliador de Expressões Matemáticas

Especificação de Requisitos e Arquitetura do Programa

CEFET-MG – Campus Timóteo · Lab. AED I · Engenharia da Computação · Prof. Luciano Moreira

Trabalho 2 · Integrantes: Isabella Senra Menezes e João Vitor Alves Silva · Novembro de 2026

1. Introdução

1.1 Objetivo

Este documento descreve o programa Avaliador de Expressões Matemáticas, desenvolvido em Java, que recebe uma expressão na notação infixa, verifica se ela é sintaticamente válida, converte-a para a notação pós-fixa (notação polonesa inversa) e calcula o seu valor numérico. O documento reúne a especificação de requisitos, as decisões de projeto, a arquitetura, os algoritmos e os testes realizados.

1.2 Escopo

O programa trabalha com expressões compostas por:

- números de um único algarismo (0 a 9);
- as cinco operações binárias: adição (+), subtração (-), multiplicação (*), divisão (/) e exponenciação (^);
- parênteses de abertura e de fechamento;
- espaços em branco, que são ignorados.

A interação com o usuário é feita por menu em console.

1.3 Restrição do enunciado

É proibido o uso de estruturas de dados prontas do Java (como Stack, ArrayList e LinkedList). Por isso, a pilha, a fila e a célula encadeada foram implementadas pelo grupo, e todo o programa usa apenas essas estruturas.

2. Requisitos

2.1 Requisitos funcionais

Código	Requisito
RF01	O programa deve ler uma expressão infixa digitada pelo usuário.
RF02	O programa deve verificar se a expressão é sintaticamente válida (quantidade e posição de operandos, operadores e parênteses) antes de qualquer conversão ou cálculo.
RF03	O programa deve converter a expressão infixa válida para a forma pós-fixa.
RF04	A conversão deve respeitar a precedência dos operadores (^ acima de * e /, que ficam acima de + e -).
RF05	Operadores de mesma precedência devem ser avaliados da esquerda para a direita, exceto o ^, que é avaliado da direita para a esquerda.
RF06	O programa deve avaliar a expressão pós-fixa e apresentar o valor numérico resultante.
RF07	O programa deve exibir a expressão pós-fixa gerada junto com o resultado.
RF08	O programa deve informar o usuário em caso de expressão inválida ou de erro de cálculo (divisão por zero), sem encerrar a execução.
RF09	O programa deve oferecer um menu que permita avaliar várias expressões na mesma execução e sair quando o usuário desejar.

2.2 Requisitos não funcionais

Código	Requisito
RNF01	O programa deve ser escrito em Java.

Código	Requisito
RNF02	Não podem ser usadas estruturas de dados prontas do Java; as estruturas são implementadas pelo grupo.
RNF03	As estruturas de dados (Pilha, Fila, Célula) devem ser genéricas e reutilizáveis, sem conhecimento sobre expressões matemáticas.
RNF04	Cada etapa do processamento (validação, conversão, avaliação) deve ficar em uma classe própria.
RNF05	O código deve ser comentado e executável em qualquer ambiente com JDK 17 ou superior.

3. Decisões de projeto e regras adotadas

Tema	Decisão
Tipo numérico	O avaliador usa double. A divisão é real: 7/2 resulta em 3,5.
Exibição do resultado	O resultado é exibido com duas casas decimais (formato %.2f).
Operandos	Somente números de um algarismo, conforme o enunciado. Números com mais de um algarismo (como 12) são rejeitados pela validação.
Sinal unário	Não é aceito. Expressões como -3+4 ou 3*-2 são inválidas.
Espaços	São ignorados em qualquer posição.
Divisão por zero	Não é erro sintático. É detectada na avaliação e gera a mensagem de erro de divisão por zero.
Associatividade	Esquerda para a direita nos operadores +, -, * e /; direita para a esquerda no operador ^.
Tratamento de erros	Erros de validação e de cálculo são lançados como exceções (IllegalArgumentException e ArithmeticException) e tratados no Main, que mostra a mensagem e volta ao menu.

4. Arquitetura

4.1 Visão geral

Todo o código está no pacote Trabalho2AED, organizado em duas camadas: as estruturas de dados genéricas (Celula, Pilha, Fila) e as classes do domínio de expressões (ValidadorExpressao, ConversorPosFixa, AvaliadorPosFixa), coordenadas pela classe Main.

4.2 Classes

Classe	Responsabilidade
Celula<T>	Nó de lista encadeada. Guarda um item e a referência para a próxima célula.
Pilha<T>	Pilha (LIFO) encadeada, com push, pop, peek e isVazia.
Fila<T>	Fila (FIFO) encadeada com célula-cabeça (sentinela), com enfileirar, desenfileirar, isVazia e toString.
ValidadorExpressao	Verifica a sintaxe da expressão infixa.
ConversorPosFixa	Converte a expressão infixa válida para pós-fixa, devolvendo uma Fila de caracteres.
AvaliadorPosFixa	Avalia a fila pós-fixa usando uma Pilha de números e devolve o valor.
Main	Menu em console; lê a expressão, chama as etapas em ordem e exibe o resultado ou a mensagem de erro.

4.3 Fluxo de execução

```
Main  ->  ConversorPosFixa.converterInfixa(expressao)
              |-> ValidadorExpressao.validarExpressao(expressao)   (se
inválida: exceção)
              |-> usa Pilha<Character> (operadores) e Fila<Character> (saída)
      ->  AvaliadorPosFixa.avaliar(fila)
              |-> usa Pilha<Double> (operandos)
      ->  exibe a pós-fixa e o resultado
```

A Pilha e a Fila são usadas por mais de uma classe, mas não dependem de nenhuma delas.

4.4 Especificação dos métodos

Pilha<T>

Método	Descrição
void push(T item)	Empilha o item no topo. Lança NullPointerException se o item for nulo.
T pop()	Remove e devolve o item do topo. Lança RuntimeException se a pilha estiver vazia.
T peek()	Devolve o item do topo sem removê-lo. Lança RuntimeException se a pilha estiver vazia.
boolean isVazia()	Indica se a pilha não possui elementos.

Fila<T>

Método	Descrição
void enfileirar(T item)	Insere o item no final. Lança NullPointerException se o item for nulo.
T desenfileirar()	Remove e devolve o item do início. Lança RuntimeException se a fila estiver vazia.
boolean isVazia()	Indica se a fila não possui elementos.
String toString()	Devolve os itens em ordem, sem removê-los.

ValidadorExpressao

Método	Descrição
boolean validarExpressao(String expressao)	Devolve true se a expressão infixa é sintaticamente válida e false caso contrário.

ConversorPosFixa

Método	Descrição
Fila<Character> converterInfixa(String expressao)	Valida e converte a expressão para pós-fixa. Lança IllegalArgumentException com a mensagem Expressão inválida! se a validação falhar.

Método	Descrição
int verificarPrecedencia(Character operador)	Devolve 1 para + e -, 2 para * e /, 3 para ^. Lança IllegalArgumentException para outro caractere.
boolean verificarOperador(Character car)	Indica se o caractere é um dos cinco operadores.

AvaliadorPosFixa

Método	Descrição
double avaliar(Fila<Character> posfixa)	Avalia a expressão pós-fixa e devolve o valor. Consome a fila.
double calcular(double a, double b, char operador)	Aplica o operador aos operandos. Lança ArithmeticException na divisão por zero e IllegalArgumentException para operador desconhecido.

5. Algoritmos

5.1 Validação sintática

A expressão é percorrida uma única vez, caractere por caractere, ignorando espaços. O algoritmo mantém duas informações: o estado atual (esperando um operando ou esperando um operador) e a quantidade de parênteses abertos.

- **Estado esperando operando** (estado inicial, e também após um operador ou um parêntese de abertura): são aceitos um dígito, que passa o estado para esperando operador, ou um parêntese de abertura, que incrementa o contador. Qualquer outro caractere torna a expressão inválida.
- **Estado esperando operador** (após um dígito ou um parêntese de fechamento): são aceitos um operador, que volta ao estado esperando operando, ou um parêntese de fechamento, desde que haja parêntese aberto, e que decrementa o contador. Qualquer outro caractere torna a expressão inválida.
- **Ao final**, a expressão é válida somente se o estado for esperando operador (a expressão não termina em operador ou em parêntese de abertura) e o contador de parênteses for zero.

Com essa regra, o exemplo do enunciado (3 (+ 5)) é rejeitado, pois o segundo parêntese de abertura aparece no estado esperando operador.

5.2 Conversão de infixa para pós-fixa

A expressão é percorrida da esquerda para a direita, usando uma Fila de saída e uma Pilha de operadores.

1. **Dígito:** é enfileirado diretamente na saída.
2. **Parêntese de abertura:** é empilhado.
3. **Parêntese de fechamento:** desempilham-se os operadores para a saída até encontrar o parêntese de abertura, que é descartado.
4. **Operador:** enquanto a pilha não estiver vazia, o topo não for um parêntese de abertura e a precedência do topo for maior ou igual à do operador lido, o topo é desempilhado para a saída. Exceção: se o operador lido for ^, nada é desempilhado por igualdade de precedência, o que garante a associatividade à direita. Em seguida, o operador lido é empilhado.
5. **Fim da expressão:** os operadores restantes na pilha são desempilhados para a saída.

A comparação entre o operador do topo da pilha e o operador lido é a sugestão feita no enunciado para detectar a precedência. A conversão é feita em uma única passagem, portanto com complexidade O(n).

Exemplos de conversão conferidos com a tabela do enunciado (letras substituídas por dígitos):

Infixa	Pós-fixa
A+B-C	AB+C-
(A+B)*(C-D)	AB+CD-*
A^B*C-D+E/F/(G+H)	AB^C*D-EF/GH+/+
((A+B)*C-(D-E))^(F+G)	AB+C*DE--FG+^
A-B/(C*D^E)	ABCDE^*/-

5.3 Avaliação da pós-fixa

A fila pós-fixa é lida do início ao fim, usando uma Pilha de números:

- **Dígito:** o seu valor numérico é empilhado (o caractere é convertido subtraindo o caractere 0).
- **Operador:** são desempilhados dois valores, sendo o primeiro a sair o operando da direita (b) e o segundo o da esquerda (a). O resultado de a operador b é empilhado.
- **Ao final**, o único valor restante na pilha é o resultado.

A ordem dos operandos é essencial para a subtração, a divisão e a exponenciação. A avaliação tem complexidade O(n).

6. Testes

Os casos abaixo foram usados para verificar o programa. Os resultados são exibidos com duas casas decimais.

6.1 Expressões válidas

Nº	Infixa	Pós-fixa	Resultado esperado	O que verifica
1	3+4*5	345*+	23,00	precedência
2	(3+4)*5	34+5*	35,00	parênteses
3	8-3-2	83-2-	3,00	associatividade à esquerda
4	1-2-3-4	12-3-4-	-8,00	resultado negativo
5	2^3^2	232^^	512,00	associatividade à direita do ^
6	2^3*4-5+6/7/(8+9)	23^4*5-67/89+/+	27,05	exemplo do enunciado, com divisão real
7	((1+2)*3-(4-5))^(1+1)	12+3*45--11+^	100,00	parênteses aninhados
8	9-2*3+4/2	923*-42/+	5,00	mistura de operadores
9	7/2*2	72/2*	7,00	divisão real, esquerda para a direita
10	2+3*(4-1)^2/3	2341-2^*3/+	11,00	^ antes de * e /
11	2^(3-3)	233-^	1,00	expoente zero
12	9^9	99^	387420489,00	número grande
13	2 + 5 * 8 / (4 + 3)	258*43+/+	7,71	exemplo do enunciado, com espaços
14	2*(3+4)*5	234+*5*	70,00	

6.2 Expressões inválidas e erros de cálculo

Infixa	Resultado esperado
(3 (+ 5))	Expressão inválida!
(3+	Expressão inválida!
3 4	Expressão inválida!
2++3	Expressão inválida!
()	Expressão inválida!
-3+4	Expressão inválida!
(vazia)	Expressão inválida!
12+3	Expressão inválida! (números de mais de um algarismo)
5/(3-3)	Erro: divisão por zero

7. Limitações conhecidas e melhorias futuras

- **Menu:** a opção do menu é lida como número inteiro; digitar uma letra nesse ponto encerra o programa. Uma melhoria é ler a opção como texto.
- **Números:** apenas números de um algarismo e sem sinal. Uma melhoria é separar a leitura da expressão em itens (tokens), o que permitiria números com vários algarismos, números decimais e sinal unário.
- **Mensagens de erro:** a validação informa apenas que a expressão é inválida. Uma melhoria é indicar a posição e o motivo do erro.
- **Testes:** os testes são feitos manualmente pelo console. Uma melhoria é automatizá-los com JUnit.

8. Referências

- Enunciado do Trabalho 2, Lab. AED I, Prof. Luciano Moreira, CEFET-MG Campus Timóteo, 12/11/2026.